

02285 AI and MAS, F26

Classical automated planning

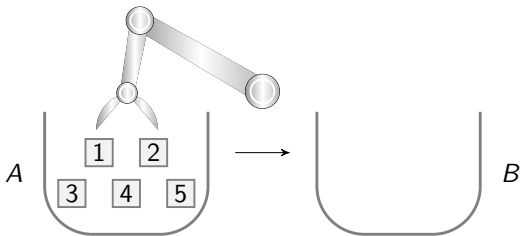
Curriculum for week 2: Sections 11.1 and 11.2 of R&N.

Automated planning also solves search problems (given initial state, goal states and actions, find a sequence of actions taking you from the initial state to a goal state). The difference is that automated planning works with logical structures on states and actions, so that an automated planner can solve *any* search problem, as long as the problem is specified using those logical structures.

Examples. Robots in changing environments, computer game AI (FEAR, Hitman, Project 007 (to appear)).

The Gripper problem

Gripper problem: Get n boxes from location A to location B using a gripper arm. A simplified version of one of the International Planning Competition (IPC) problems (1998–).



Initial state: The above (all boxes in A).

Goal state: All boxes in B .

Actions:

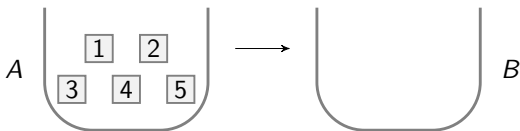
Pick(x, y): pick up box x at location y with gripper arm;

Drop(x, y): drop box x from arm at location y ;

Move(x, y): move gripper arm from location x to location y .

Simplified Gripper problem

Simplified Gripper problem: The arm is abstracted away.



Initial state: The above (all boxes in A).

Goal state: All boxes in B .

Actions. $MoveAB(x)$: move box number x from location A to location B ; $MoveBA(x)$: move box x from B to A .

What is the *state space* of this planning problem?

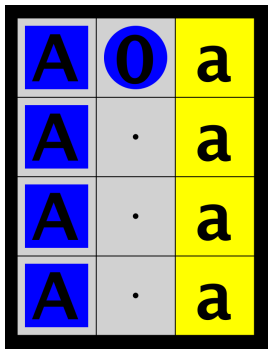
Simplified Gripper problem cont'd

The simplified Gripper problem as a **pure search problem**:

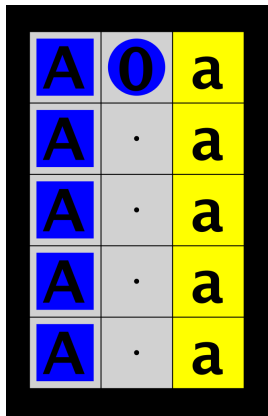
- Build **state space** (a graph).
- **Initial state**: all boxes in A .
- **Goal state**: all boxes in B .
- Find a path from the initial state to the goal state in the state space (use e.g. BFS or DFS).

What is the size of the state space, in Θ -notation, for an instance with n boxes?

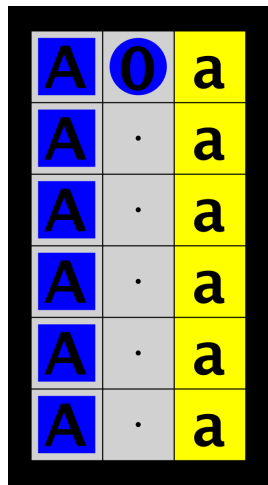
A Gripper-like problem in the setting of the hospital domain



gripper04

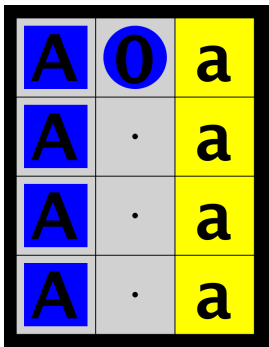


gripper05



gripper06

BFS on the Gripper-like problem



level	generated nodes
gripper04	3,755
gripper05	26,792
gripper06	184,326
gripper07	1,221,887
gripper08	7,956,581

Every time we add one box, the number of generated nodes is multiplied by ≈ 7 . Why is that?

Heuristics for Gripper

Conclusion on the above: To make planning feasible even in the simplest domains, it is necessary to have **efficient heuristics**.

Designing an efficient heuristic for the (simplified) Gripper problem is not difficult.

How?

Which search strategy would work well using this heuristic?

What will be the complexity of the resulting algorithm?

Informed search on the Gripper-like problem

Two heuristics:

- $h_{gc}(s)$ is the **goal-count heuristic**, that is, $h_{gc}(s)$ = the number of misplaced boxes.
- $h_w(s)$ is a heuristic for the warmup assignment (the teacher's solution).

Number of generated nodes for the various search strategies and heuristics:

level	BFS	A^*/h_{gc}	greedy/ h_{gc}	A^*/h_w	greedy/ h_w
gripper04	3,755	2,148	92	64	63
gripper05	26,792	7,768	127	83	70
gripper06	184,326	37,924	328	106	78
gripper07	1,221,887	140,268	246	126	113
gripper08	7,956,581	477,701	289	152	118
gripper10	-	7,196,983	364	190	154
gripper20	-	-	1,169	397	324
gripper50	-	-	3,165	1,010	865

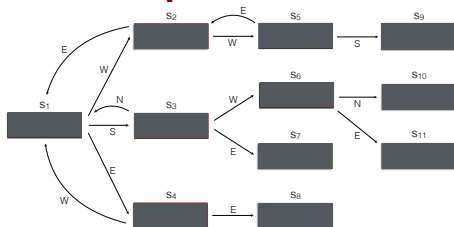
Weaknesses of problem-solving by searching

- States of the system are represented as **atoms** (nodes) with no internal structure.
- Without heuristics we are simply doing “blind search” (zero intelligence).
- Atomic graph representation only allows for problem-specific heuristics.
- Consequence: Each new domain requires a new problem specification (ACTIONS, RESULT, GOAL-TEST, etc.) and a new handcrafted problem-specific heuristic. Not very intelligent...

Solution: Give **structure** to the states of the state space in a uniform and domain-independent way allowing one to automatically generate heuristics in a uniform way. This is done in the field of **automated planning**.

Finding the sweet spot

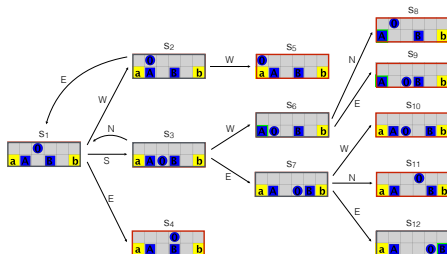
No structure on states:
blind search, computationally *infeasible*.



Sweet spot: computationally *feasible* **and** *automatic* heuristics.

AUTOMATED PLANNING: Generic structure on states, generic heuristics

Use full state structure:
heuristic search using *manual* heuristics.



State representation example

How to give structure to states in the Gripper example? We can e.g. use **predicate logic (first-order logic)** (cf. R&N Chapter 8).

Initial state:

$$In(1, A) \wedge In(2, A) \wedge \dots \wedge In(n, A)$$

Goal state:

$$In(1, B) \wedge In(2, B) \wedge \dots \wedge In(n, B).$$

In is a binary **predicate (predicate symbol)**, $1, 2, \dots, n$, A and B are **constants**.

Result of performing the action $MoveAB(2)$ in the initial state:

$$In(1, A) \wedge In(2, B) \wedge In(3, A) \wedge \dots \wedge In(n, A).$$

Basic notions from (function-free) first-order logic

- **n -ary predicate:** A symbol denoting an n -ary relation. Takes n arguments. **Examples:** *In* is a 2-ary predicate. **Conventions:** We use capitalised words for predicate like *In*, *Move*, *Buy*, *Kill*.
- **Variables:** As usual. **Conventions:** We use small letters like x , y , z , a , b .
- **Constants:** A symbol denoting an object. **Conventions:** As for predicates.
- **Terms:** Constants and variables (we don't use function symbols).
- **Atom:** An n -ary predicate applied to n terms. **Examples:** $In(x, y)$, $In(x, 1)$, $In(1, 2)$.
- **Literal:** An atom (**positive literal**) or its negation (**negative literal**). **Examples:** $In(x, y)$, $\neg In(x, y)$.
- **Ground atom/literal:** All arguments are constants.

The logic has equality, that is, $=$ is among the predicates (infix notation).

Planning using logic

Advantage of structural representation of states: Allows us to define general and domain-independent heuristics in terms of these representations.

Example of a domain-independent heuristic:

$h(s)$ = the number of goal literals that *do not* hold in s .

In the Gripper problem the state

$$s = In(1, A) \wedge In(2, B) \wedge In(3, A) \wedge \cdots \wedge In(n, A)$$

has $h(s) = n - 1$.

Note: This gives exactly the goal count heuristic, counting the number of misplaced boxes. Also works with other goal configurations, e.g. all even numbered boxes in A , all odd-numbered in B .

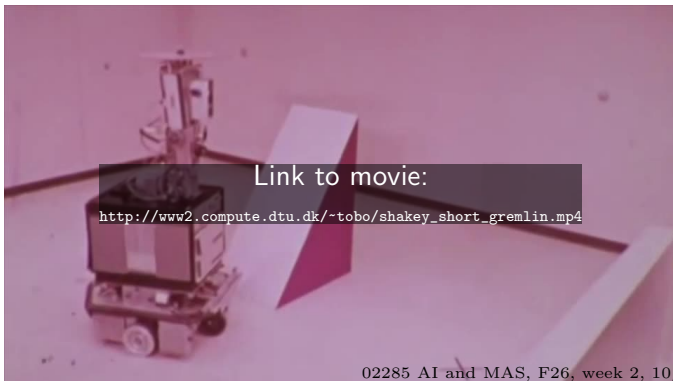
Intermediate summary

- Planning as pure search: combinatorial explosion even for the simplest problems—unless we have efficient domain-specific heuristics.
- Avoiding domain-specific heuristics for each new planning problem: put logical structure on states.

Example. Assume our goal is to get milk, bananas, and a cordless drill. We would get closer to our goal by starting out buying milk than e.g. by buying coffee or exotic underwear. This is immediately captured by planning using logical representations of states, but not without structure on the states.

A language for automated planning: PDDL

- **PDDL** stands for *Planning Domain Definition Language*.
- De facto standard for representing planning problems using (function-free first-order) logic.
- PDDL has a programming language-like syntax, but we'll use a more logically oriented syntax here (as does the textbook).
- PDDL is an extension of the **STRIPS** language (*STanford Research Institute Problem Solver*), developed for Shakey the Robot.



Link to movie:

http://www2.compute.dtu.dk/~tobo/shakey_short_gremlin.mp4

From search to PDDL planning (+ PDDL states)

	search	PDDL
states	graph nodes	conjunction of atoms
actions	graph edges	action schemas
goal	graph node(s)	conjunction of literals
plan	path	sequence of actions

Definition (State)

In PDDL, a **state** is a conjunction (or set) of ground atoms.

Examples of states:

$$In(1, A) \wedge In(2, B) \wedge In(3, A). \quad (1)$$

$$At(Shop) \wedge Have(Milk) \wedge Have(Exotic Underwear). \quad (2)$$

We work under the **closed-world assumption (CWA)**: everything not explicitly claimed to be true by the state description is assumed false.

Example. In the state described by (1) e.g. the following holds:

$$\neg In(1, B) \text{ and } \neg In(2, A).$$

Action schemas in PDDL

In PDDL, **actions** are represented by **action schemas (operators)**.

Example:

ACTION: $MoveAB(x)$

PRECONDITION: $Box(x) \wedge In(x, A)$

EFFECT: $In(x, B) \wedge \neg In(x, A)$

Definition (Action schemas)

Action schemas (operators) consist of three elements:

1. **Action name.** A predicate applied to variables. **Examples:** $MoveAB(x)$, $Buy(x, y)$, $Pick(x, y)$.
2. **Precondition.** A conjunction of literals. **Examples:** $Box(x) \wedge In(x, A)$, $At(y) \wedge Sells(y, x)$, $At(y) \wedge In(x, y) \wedge Free$.
3. **Effect.** A conjunction of literals. **Examples:** $In(x, B) \wedge \neg In(x, A)$, $Have(x)$, $Carry(x) \wedge \neg In(x, y) \wedge \neg Free$.

Constraint: Preconditions and effects may only contain variables occurring in the action name.

Goals, planning domain and planning problems

Definition (Goal)

A **goal** is a conjunction of literals. **Ex:** $In(1, B) \wedge In(2, B) \wedge In(3, B)$.

Definition (Planning domains and planning problems)

A **planning domain** is a set $\mathcal{A}$ of action schemas.

A **planning problem** is a triple $(\mathcal{A}, s_0, g)$ where:

1. $\mathcal{A}$ is a planning domain (a set of action schemas).
2. s_0 is a state called the **initial state**.
3. g is a goal (a conjunction of literals).

Example of a planning problem (simplified Gripper).

1. Action schemas:

ACTION: $MoveAB(x)$

PRECONDITION: $Box(x) \wedge In(x, A)$

EFFECT: $In(x, B) \wedge \neg In(x, A)$

ACTION: $MoveBA(x)$

PRECONDITION: $Box(x) \wedge In(x, B)$

EFFECT: $In(x, A) \wedge \neg In(x, B)$

2. **Initial state:** $Box(1) \wedge \dots \wedge Box(n) \wedge In(1, A) \wedge \dots \wedge In(n, A)$.

3. **Goal:** $In(1, B) \wedge \dots \wedge In(n, B)$.

Ground actions

Definition (Ground actions)

Ground action: An action obtained by instantiating the variables of an action schemas to a choice of constants.

Example. Action schema:

ACTION: $MoveAB(x)$

PRECONDITION: $Box(x) \wedge In(x, A)$

EFFECT: $In(x, B) \wedge \neg In(x, A)$

A couple of ground actions (instantiations) of the schema:

ACTION: $MoveAB(1)$

PRECONDITION: $Box(1) \wedge In(1, A)$

EFFECT: $In(1, B) \wedge \neg In(1, A)$

ACTION: $MoveAB(2)$

PRECONDITION: $Box(2) \wedge In(2, A)$

EFFECT: $In(2, B) \wedge \neg In(2, A)$

The ground actions are the executable actions in the domain.

Applicability of a ground action

Definition (Applicability)

A ground action a is called **applicable** in a state s if its precondition is logically entailed by s . In logical notation:

$$s \models \text{PRECONDITION}(a).$$

Equivalently: All positive literals in $\text{PRECONDITION}(a)$ occur in s and none of the negative literals in $\text{PRECONDITION}(a)$ occur in s .

Example. In the state $\text{Box}(1) \wedge \text{Box}(2) \wedge \text{In}(1, A) \wedge \text{In}(2, B)$ the action $\text{MoveAB}(1)$ is applicable, but not $\text{MoveAB}(2)$.

How many instantiations of $\text{MoveAB}(x)$ are applicable in the initial state $\text{Box}(1) \wedge \dots \wedge \text{Box}(n) \wedge \text{In}(1, A) \wedge \dots \wedge \text{In}(n, A)$?

The result of executing an action in a state

Definition (Add and delete lists)

Let a denote a ground action.

Add list of a : The set of positive literals in $\text{EFFECT}(a)$, denoted $\text{ADD}(a)$. The atoms that come to hold when a is executed.

Delete list of a : The set of negative literals in $\text{EFFECT}(a)$, denoted $\text{DEL}(a)$. The atoms that stop holding when a is executed.

Example. Consider the following ground action $a = \text{MoveAB}(1)$:

ACTION: $\text{MoveAB}(1)$

PRECONDITION: $\text{Box}(1) \wedge \text{In}(1, A)$

EFFECT: $\text{In}(1, B) \wedge \neg \text{In}(1, A)$

We have $\text{ADD}(a) = \{\text{In}(1, B)\}$ and $\text{DEL}(a) = \{\text{In}(1, A)\}$.

Definition (Result function)

The **result** of executing an applicable action a in a state s is given by (identifying states with sets of literals = **set semantics**):

$$\text{RESULT}(s, a) = (s - \text{DEL}(a)) \cup \text{ADD}(a)$$

Solutions to planning problems

Definition (Solution to a planning problem)

A **solution** to a planning problem $(\mathcal{A}, s_0, g)$ is a plan (an action sequence) leading from the start state s_0 to a state satisfying the (existential closure of the) goal g .

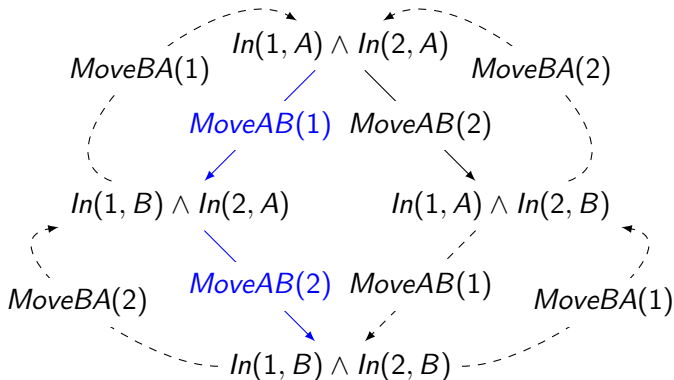
More precisely: A sequence of ground actions $a_1, a_2, \dots, a_n$ from $\mathcal{A}$ satisfying:

- a_1 is applicable in the initial state s_0 .
- For all $i = 2, \dots, n$: a_i is applicable in the result of executing the sequence of actions $a_1, \dots, a_{i-1}$ in the initial state s_0 .
- The state resulting from executing the sequence of actions $a_1, \dots, a_n$ in the initial state s_0 satisfies the (existential closure of the) goal g .

Solutions can be computed e.g. with GRAPH-SEARCH (BFS, DFS, A^* , greedy best-first search), where the states are conjunctions (or, equivalently, sets) of atoms.

Graph-Search on simplified Gripper

The full state space as explored by BFS GRAPH-SEARCH with initial state $s_0 = In(1, A) \wedge In(2, A)$ and goal $g = In(1, B) \wedge In(2, B)$. The *Box* predicate is **static** (can never change), so is omitted.



An optimal solution is highlighted in blue.

Domain-independent heuristics in automated planning

- We still need an accurate **heuristic** for progression planning to be practical (consider e.g. the Gripper problem).
- The logical structure on the states allows us to define a number of completely domain-independent heuristics. We already considered one very simple such: count the number of unsatisfied goal literals (called the **goal count heuristic**).
- The field of automated planning has developed a lot of quite advanced domain-independent heuristics that analyse the logical structure of the action schemas to extract a sensible heuristic function. This normally works by relaxing the problem via simplifications of the schemas (e.g. ignoring (some of) the preconditions or ignoring negative effects). Then recall that any problem relaxation induces an admissible heuristic. Much more on this next week...

Planning problem example: Milk and bananas

Consider a world containing the following two action schemas:

ACTION: $Buy(x, y)$ (buy item x at location y)

PRECONDITION: $Buyable(x) \wedge Place(y) \wedge At(y) \wedge Sells(y, x)$

EFFECT: $Have(x)$

ACTION: $Go(x, y)$

PRECONDITION: $Place(x) \wedge Place(y) \wedge At(x)$

EFFECT: $At(y) \wedge \neg At(x)$

Assume we are given the following initial state and goal:

Init state: $Buyable(Milk) \wedge Buyable(Bananas) \wedge Buyable(Drill) \wedge Place(Home) \wedge Place(Netto) \wedge Place(ToolShop) \wedge At(Home) \wedge Sells(Netto, Milk) \wedge Sells(Netto, Bananas) \wedge Sells(ToolShop, Drill)$.

Goal: $At(Home) \wedge Have(Milk) \wedge Have(Bananas) \wedge Have(Drill)$.

A possible **solution** (plan) leading from start to goal is:

$Go(Home, Netto), Buy(Milk, Netto), Buy(Bananas, Netto),$
 $Go(Netto, ToolShop), Buy(Drill, ToolShop), Go(ToolShop, Home)$.

Designing planning domains and problems

Observations:

- Planning domains and problems can be designed in many **distinct**—but equivalent—ways.
- Designing them efficiently is **non-trivial**.
- **Bad domain design** can potentially **destroy** the advantage over planning as pure search: Take one unique literal per state of the state space of the problem.